udp FACULTAD DE
INGENIERÍA Y CIENCIAS

Estructura de Datos: Laboratorio 2

Alvaro Valdebenito
Profesor: Yerko Ortiz
Ayudante: Diego Banda
Sección 4

26 de abril 2024

Índice

1. Introducción **3**

2. Desarrollo **4**

2.1. Declaración del la LinkedList 4
2.2. Métodos para el Reproductor de música 5
2.3. Métodos para las Playlist 14
2.4. Método Main 16

3. Conclusión **17**

1. Introducción

A continuación veremos la solución al problema de no poder de pagar un reproductor de música premium. Para esto se crea uno propio utilizando nuestro conocimiento sobre estructuras lineales.

Para esto se tiene como objetivo diseñar e implementar un sistema de reproducción de música utilizando el lenguaje de programación de java. Para esto se utilizarán distintas **TDA** (Tipo de Dato Abstracto), como pueden ser **LinkedList, Stack** y **Queue**. De forma que se puedan gestionar de mejor manera los métodos del Reproductor, junto a un Menú interactivo, para mayor facilidad en su uso.

2. Desarrollo

Ahora veremos la implementación del código, junto a una breve explicación a los distintos métodos realizados en él, para una mayor comprensión.

2.1. Declaración del la LinkedList

```java
public class Lab2{

    class Cancion {
        public String nombre ;
        public int duracion ; // en segundos
        public String artista ;
        public Cancion next;

        public Cancion ( String nombre , int duracion , String artista ){
            this. nombre = nombre ;
            this. duracion = duracion ;
            this. artista = artista ;
            this.next = null;
            }
    }

    private Cancion head;
    private Stack<Cancion> historialDeReproduccion;
    private LinkedList<Queue<Cancion>> bibliotecaDeReproduccion;
    private int counter;

    public Lab2(){
        this.head = null;
        historialDeReproduccion = new Stack<>();
        bibliotecaDeReproduccion = new LinkedList<>();
        counter = 0;
    }
```

Figura 1: Class Canción.

Se utiliza la clase **Cancion** con la finalidad de crear una LinkedList en la cual se puedan manjar distintos tipos de datos dentro de ella, como pueden ser en este caso: **Nombre, duracion, artista**, además de un **next** para poder moverse dentro de ella. Se realizan los constructores correspondientes para la clase Cancion, junto al constructor de la clase principal (Lab2), el cual posee distintos atributos para la funcionalidad del código más adelante.

2.2. Métodos para el Reproductor de música

```java
public void agregarCancion(String nombre, int duracion, String artista)
{
    Cancion newSong = new Cancion(nombre, duracion, artista);

    if(this.head == null){
        this.head = newSong;
    }

    else{
        Cancion aux = this.head;
        while(aux.next != null)
        {
            aux = aux.next;
        }
        aux.next = newSong;
    }
}
```

Figura 2: Método para Agregar Canción.

En este método se reciben 3 parámetros los cuales son las características de la canción, se declara la variable *newSong* con dichos parámetros y verificamos si está vacía, en caso de que si lo esté, añadimos la canción al head de la LinkedList, en caso contrario, se crea una variable auxiliar la cual apuntará al head.

```java
public void eliminarCancion(String nombre, String artista)
{
    Cancion prev = null;
    Cancion current = this.head;

    while(current != null)
    {
        if(current.nombre.equals(nombre) && current.artista.equals(artista))
        {
            if(prev == null)
            {
                this.head = current.next;
            }

            else
            {
                prev.next = current.next;
            }
            System.out.println("Cancion: [" + current.nombre + " - " + current.artista + "] eliminada.");
            return;
        }
        prev = current;
        current = current.next;

    }
    System.out.println("Canción no encontrada en la cola de reproduccion.");
}
```

Figura 3: Método para Eliminar Canción.

Se reciben los parámetros **nombre** y **artista**, y creamos las variables *prev* y *current* con el fin de recorrer la LinkedList siempre y cuando esta no sea nula, y con ayuda del método *.equals* podemos comparar si el nombre y artista son equivalentes a los que queremos comparar, de esta forma, si la encuentra, es eliminada.

```java
public void reproducir()
{
    if(this.head == null)
    {
        System.out.println("No se encuentran canciones en la cola de reproduccion");
        return;
    }

    Cancion playSong = this.head;
    this.head = this.head.next;

    historialDeReproduccion.push(playSong);
    System.out.println("Reproduciendo: " + playSong.nombre + " - " + playSong.artista);

}
```

Figura 4: Método para Reproducir Canción.

En este método reproducimos las canciones que se hayan agregado anteriormente, para esto se crea la variable *playSong* la cual tomará el valor del head. Luego hacemos que el head, tome el valor al cual apunta, para así eliminarlo. A continuación se añade a un Stack de **Historial de Reproducciones** declarado en los atributos de Lab2 definidos anteriormente.

```java
public void imprimirHistorialDeReprodiccion()
{
    System.out.println("Historial de Reproduccion: ");
    Stack<Cancion> temp = new Stack<>();
    while(!historialDeReproduccion.isEmpty())
    {
        Cancion song = historialDeReproduccion.pop();
        temp.push(song);
        System.out.println(song.nombre + " - " + song.artista);
    }

    while(!temp.isEmpty())
    {
        historialDeReproduccion.push(temp.pop());
    }
}
```

Figura 5: Método para Imprimir el Historial de Reproducción.

Como se mostró en el método anterior, utilizamos un Stack llamado **historialDeReproduccion** y utilizamos la función .push para añadir los elementos. Se crea un Stack temporal, y con ayuda del ciclo while, e iremos vaciando el historial, mientras vamos llenando nuestro temporal creado, de esta forma, no borramos los datos que almacenamos anteriormente, finalmente vaciamos el temporal, para nuevamente llenar el Stack principal con los datos que tenía al comienzo.

```java
public void buscarCancion(String buscar)
{
    boolean aux = false;
    Cancion current = this.head;

    while(current != null)
    {
        if(current.nombre.equals(buscar) || current.artista.equals(buscar))
        {
            System.out.println("Cancion encontrada en la cola de reproduccion: [" + current.nombre + " - " + current.artista + " (" + current.duracion + "s)]");
            aux = true;
        }
        current = current.next;
    }

    for(int i=0; i<historialDeReproduccion.size();i++)
    {
        Cancion song = historialDeReproduccion.get(i);
        if(song.nombre.equals(buscar) || song.artista.equals(buscar))
        {
            System.out.println("Cancion encontrada en el historial de reproduccion: " + song.nombre + " - " + song.artista + " (" + song.duracion + ")");
            aux = true;
        }
    }

    if(!aux)
    {
        System.out.println("No se encontro ninguna cancion con dicho Nombre/Artista.");
    }
}
```

Figura 6: Método para Buscar Canción.

En este método declaramos dos variables, un *aux* que inicializamos en false, y un *current* que será igual al head. Luego se recorre la LinkedList con ayuda del ciclo While hasta que no sea nulo. A continuación buscamos en la cola de reproducción con la ayuda del método .equals, si esta es encontrada, cambiamos el valor de *aux* a true. Realizamos el mismo procedimiento para el historial de reproducción. Si no encuentra la canción, gracias al *aux* lo sabremos.

```java
public void estadisticasDeReproduccion()
{
    if (historialDeReproduccion.isEmpty())
    {
        System.out.println("No se han reproducido canciones aun.");
        return;
    }

    String[] songs = new String[historialDeReproduccion.size()];
    int[] reproducciones = new int[historialDeReproduccion.size()];
    int duracionTotal = 0;
    int aux = 0;

    for (int i = 0; i < historialDeReproduccion.size(); i++)
    {
        Cancion song = historialDeReproduccion.get(i);
        duracionTotal += song.duracion;
        String claveCancion = song.nombre + " - " + song.artista;
        boolean encontrada = false;

        for (int j = 0; j < aux; j++)
        {
            if (songs[j].equals(claveCancion))
            {
                reproducciones[j]++;
                encontrada = true;
                break;
            }
        }

        if (!encontrada)
        {
            songs[aux] = claveCancion;
            reproducciones[aux] = 1;
            aux++;
        }
    }

    System.out.println("Canciones mas reproducidas:");
    for (int i = 0; i < aux; i++)
    {
        System.out.println(songs[i] + ": " + reproducciones[i] + " reproducciones");
    }
```

Figura 7: Método para ver las Estadísticas (Primera parte).

Este método lo dividiremos en dos partes. Primero verificamos si el historial de reproducción está vacío, luego se crean arreglos: *songs* el cual almacena las características de las canciones, y *reproducciones* el cual va a almacenar el número de reproducciones de cada canción. Además se inicializa la variable *duracionTotal*, para ver el tiempo total entre las canciones reproducidas. Luego, gracias a la ayuda de un bucle for recorremos el historial de

reproducciones, e incrementamos la *duracionTotal*, en base a los segundos de cada canción y la guardamos en la variable *claveCancion*. Luego recorremos otro ciclo for, en el cual vamos comparando con la ayuda del método *.equals*, si la canción es encontrada, incrementa los contadores, en caso contrario, gracias al método *aux*, establece el contador en 1 para esa canción, y luego incrementa *aux*. Para finalmente imprimir las canciones más reproducidas.

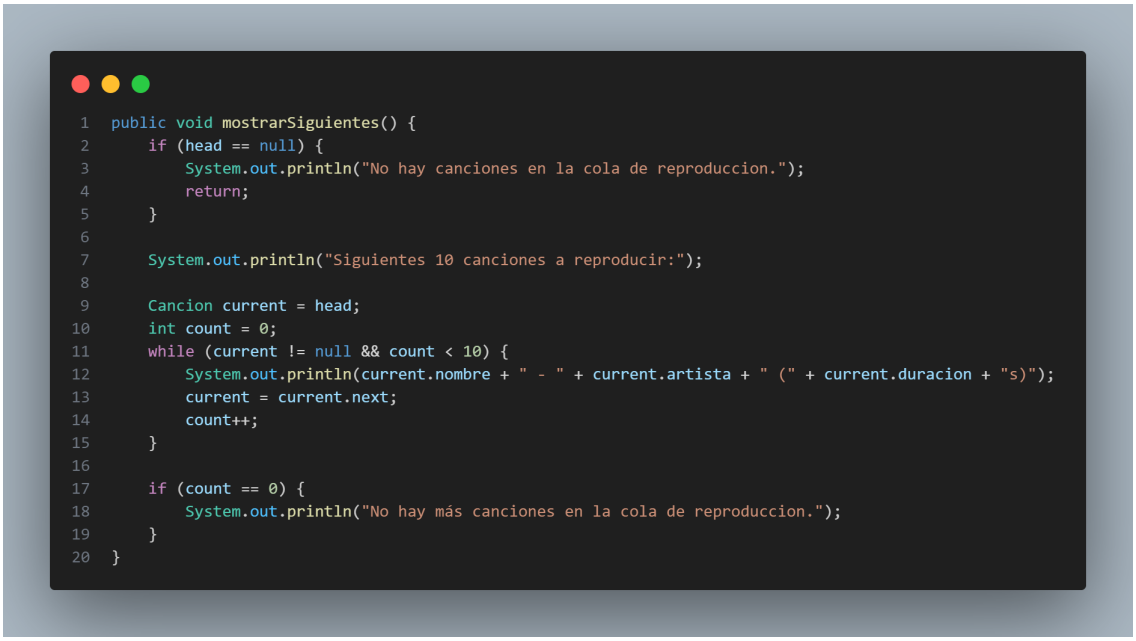
```
    System.out.println("Duracion total de reproduccion:");
    if (duracionTotal >= 3600)
    {
        int horas = duracionTotal / 3600;
        int minutos = (duracionTotal % 3600) / 60;
        int segundos = duracionTotal % 60;
        System.out.println(horas + " horas, " + minutos + " minutos, " + segundos + " segundos");
    }

    else if (duracionTotal >= 60)
    {
        int minutos = duracionTotal / 60;
        int segundos = duracionTotal % 60;
        System.out.println(minutos + " minutos, " + segundos + " segundos");
    }

    else
    {
        System.out.println(duracionTotal + " segundos");
    }
}
```

Figura 8: Método para ver las Estadísticas (Segunda parte).

Para la 2da parte del código, gracias a la variable *duracionTotal* la cual almacenó el tiempo en segundos, utilizamos operaciones matemáticas para así poder transformarlo, ya sea en horas, minutos o segundos, dependiendo del tiempo de este.



```java
public void mostrarSiguientes() {
    if (head == null) {
        System.out.println("No hay canciones en la cola de reproduccion.");
        return;
    }

    System.out.println("Siguientes 10 canciones a reproducir:");

    Cancion current = head;
    int count = 0;
    while (current != null && count < 10) {
        System.out.println(current.nombre + " - " + current.artista + " (" + current.duracion + "s)");
        current = current.next;
        count++;
    }

    if (count == 0) {
        System.out.println("No hay más canciones en la cola de reproduccion.");
    }
}
```

Figura 9: Método para Mostrar las Canciones en la Cola.

En este método queremos mostrar las canciones en la cola de reproducción, para esto creamos las variables *current* y un *contador*. Con ayuda de un ciclo while, recorremos la LinkedList hasta que sea nula, o hasta que llegue a 10, y aumenta el contador. Si el contador no aumenta, verificamos que no hay canciones en la cola de reproducción.

```java
public void mostrarUltimas10CancionesHistorial()
{
    System.out.println("Últimas 10 canciones del historial de reproducción:");
    Stack<Cancion> tempStack = new Stack<>();
    int count = 0;

    while (!historialDeReproduccion.isEmpty() && count < 10) {
        Cancion song = historialDeReproduccion.pop();
        tempStack.push(song);
        count++;
    }

    while (!tempStack.isEmpty()) {
        Cancion song = tempStack.pop();
        System.out.println(song.nombre + " - " + song.artista);
    }

    while (!tempStack.isEmpty()) {
        historialDeReproduccion.push(tempStack.pop());
    }
}
```

Figura 10: Método para Mostrar las Últimas 10 Canciones en el Historial.

Para este método, se crea un Stack *temporal* y un *contador*, luego recorremos con la ayuda de un ciclo while el historial de reproducción si no está vacío y el contador sea menor que 10. Luego creamos la variable *song* la cual tomará el valor de la canción eliminada con la función *.pop*, para luego ser agregada al Stack *temporal* por el método *.push*, y finalmente aumentando el contador. Después se vacía el Stack *temporal* de forma que quede guardado nuevamente en el Stack de *Historial de reproducción*, para así no modificarlo, ni eliminarlo.

2.3. Métodos para las Playlist

```java
public void crearColaDeReproduccion()
 {
    Queue<Cancion> newCola = new LinkedList<>();
    bibliotecaDeReproduccion.add(newCola);
    System.out.println("Cola de reproduccion #" + counter + " creada." );
   counter++;
}
```

Figura 11: Método para crear una Cola de Reproducción.

En este método lo que hacemos es crear una cola de reproducción, o bien, una PlayList. Para esto, vimos anteriormente en los atributos de Lab2 una **Biblioteca De Reproducción**, la cual es una LinkedList que contiene Colas de canciones y un **Counter**, el cual inicializa en 0. Entonces, se crea la variable *newCola* y lo añadimos a la Biblioteca de Reproducción con el índice del contador para trabajar con los siguientes métodos.

```java
public void eliminarColaDeReproduccion(int num)
{
    if(num >= 0 && num < bibliotecaDeReproduccion.size())
    {
        bibliotecaDeReproduccion.remove(num);
        counter--;
        System.out.println("Cola de Reproduccion #" + num + " eliminada.");
    }

    else{
        System.out.println("La Cola de Reproduccion #" + num + " no se encuentra creada.");
    }
}
```

Figura 12: Método para Eliminar una Cola de Reproducción.

Para eliminar la Playlist, se recibe un parámetro el cual nos indica el número de la Playlist que queremos eliminar. Si la encuentra, utilizamos la función *.remove* en la Biblioteca de Reproducción y se elimina.

```java
public void agregarCancionColaReproduccion(int num, String nombre, int duracion, String artista)
{
    if(num >= 0 && num < bibliotecaDeReproduccion.size())
    {
        Queue<Cancion> colaReproduccion = bibliotecaDeReproduccion.get(num);
        Cancion newSong = new Cancion(nombre, duracion, artista);
        colaReproduccion.add(newSong);
        System.out.println("Cancion agregada a la Cola de Reproduccion #" + num);
    }

    else{
        System.out.println("No se encuentra la cola de reproduccion #" + num);
    }
}
```

Figura 13: Método para Agregar Canción a una Cola de Reproducción.

Se reciben los parámetros con las características de la canción, y el índice de la Playlist en la cual se quiere agregar una canción. Se crea la variable *colaReproduccion* para obtener la cola de reproducción específica en la Biblioteca de Reproducción, para luego añadir la canción a dicha cola.

```java
public void reproducirPlaylist(int num)
{
    if(num >= 0 && num < bibliotecaDeReproduccion.size())
    {
        Queue<Cancion> playList = bibliotecaDeReproduccion.get(num);
        if(!playList.isEmpty())
        {
            System.out.println("Reproduciendo PlayList Seleccionada.");
            Cancion song = playList.poll();
            System.out.println("Reproduciendo: [" + song.nombre + " - " + song.artista + "(" + song.duracion + "s)]");
            historialDeReproduccion.push(song);
        }
        else{
            System.out.println("La PlayList seleccionada esta vacia.");
        }
    }
    else{
        System.out.println("La PlayList #" + num + " no existe");
    }
}
```

Figura 14: Método para Reproducir una Cola de Reproducción.

Recibimos el índice de la cola de reproducción la cual queremos reproducir y verificamos que exista, en ese caso buscamos la Cola de Reproducción con ese índice, y verificamos que no esté vacía, para luego extraer la primera canción de la lista *song* utilizando el método *.poll*, para luego añadir al historial de reproducción.

2.4. Método Main

```java
public static void main(String[] args) {
    Lab2 menu = new Lab2();
    Scanner scanner = new Scanner(System.in);

    int opcion;
    do {
        System.out.println("\n=== Menú ===");
        System.out.println("1. Agregar Cancion");
        System.out.println("2. Eliminar Cancion");
        System.out.println("3. Historial de Reproduccion");
        System.out.println("4. Reproducir Cancion");
        System.out.println("5. Buscar Cancion");
        System.out.println("6. Reproducir Playlist");
        System.out.println("7. Crear Cola de Reproduccion Personalizada (Playlist)");
        System.out.println("8. Agregar Cancion a Cola de Reproduccion (Playlist)");
        System.out.println("9. Eliminar Cola de Reproduccion (Playlist)");
        System.out.println("10. Estadisticas de Reproduccion");
        System.out.println("11. Mostrar siguientes 10 canciones a reproducir");
        System.out.println("12. Mostrar siguientes 10 canciones en el historial");
        System.out.println("13. Salir");
        System.out.print("Ingrese una opcion: ");
        opcion = scanner.nextInt();

        switch (opcion) {
            case 1:
                System.out.print("Ingrese el nombre de la cancion: ");
                String nombre = scanner.next();
                System.out.print("Ingrese la duracion de la cancion (en segundos): ");
                int duracion = scanner.nextInt();
                System.out.print("Ingrese el nombre del artista: ");
                String artista = scanner.next();
                menu.agregarCancion(nombre, duracion, artista);
                break;
            case 2:
                System.out.print("Ingrese el nombre de la cancion a eliminar: ");
                String nombreEliminar = scanner.next();
                System.out.print("Ingrese el nombre del artista de la cancion a eliminar: ");
                String artistaEliminar = scanner.next();
                menu.eliminarCancion(nombreEliminar, artistaEliminar);
                break;
            case 3:
                menu.imprimirHistorialDeReprodiccion();
                break;
            case 4:
                menu.reproducir();
                break;
            case 5:
                System.out.print("Ingrese el nombre o el artista de la cancion a buscar: ");
                String buscar = scanner.next();
                menu.buscarCancion(buscar);
                break;
            case 6:
                System.out.print("Ingrese el numero de la playlist a reproducir: ");
                int numPlaylist = scanner.nextInt();
                menu.reproducirPlaylist(numPlaylist);
                break;
            case 7:
                menu.crearColaDeReproduccion();
                break;
            case 8:
                System.out.print("Ingrese el numero de la cola de reproduccion: ");
                int numCola = scanner.nextInt();
                System.out.print("Ingrese el nombre de la cancion a agregar: ");
                String nombreCancion = scanner.next();
                System.out.print("Ingrese la duracion de la cancion (en segundos): ");
                int duracionCancion = scanner.nextInt();
                System.out.print("Ingrese el nombre del artista: ");
                String artistaCancion = scanner.next();
                menu.agregarCancionColaReproduccion(numCola, nombreCancion, duracionCancion, artistaCancion);
                break;
            case 9:
                System.out.print("Ingrese el numero de la cola de reproduccion a eliminar: ");
                int numColaEliminar = scanner.nextInt();
                menu.eliminarColaDeReproduccion(numColaEliminar);
                break;
            case 10:
                menu.estadisticasDeReproduccion();
                break;
            case 11:
                menu.mostrarSiguientes();
                break;
            case 12:
                menu.mostrarUltimas10CancionesHistorial();
                break;
            case 13:
                System.out.println("Saliendo del programa...");
                break;
            default:
                System.out.println("Opción invalida, por favor ingrese un número entre 1 y 13.");
        }
    } while (opcion != 13);

    scanner.close();
  }
}
```

Figura 15: Método main.

Finalmente, se crea el método main de manera interactiva, en el cual se implementa un do while, en el cual el usuario puede seleccionar las opciones deseadas, gracias a la ayuda del switch, el cual verá caso a caso.

3. Conclusión

Finalmente, se cumple el objetivo principal, el cual era poder crear un Reproductor de Música en base a los distintos Tipos de Datos Abstractos, si bien hubo métodos en los cuales no requería mucho código, hubo otros en los cuales había que pensar bien de qué manera debían ser implementados, como por ejemplo la LinkedList de colas para trabajar con una biblioteca, o por ejemplo de que forma poder obtener las estadísticas de las canciones, que a mi parecer fue bien trivial. Gracias a este laboratorio, aprendí de mejor forma cómo trabajar con LinkedList, y cómo trabajar no solo con números dentro de ellas, sino con información de distintos tipos a grandes rasgos.